

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет «Программной инженерии и компьютерной техники»

Информационная безопасность

Лабораторная работа №1

Выполнил:

Григорьев Давид Владимирович
Группа: Р3415

Проверил:

Маркина Татьяна Анатольевна

Санкт-Петербург, 2026

Содержание

1	Цель работы	3
2	Описание проекта и API	3
2.1	Стек технологий	3
2.2	Endpoints	3
2.3	Примеры запросов	3
3	Реализованные меры защиты	4
3.1	Защита от SQL-инъекций	4
3.2	Защита от XSS	4
3.3	Защита аутентификации	4
4	Ход работы и тестирование	4
4.1	Реализация API	4
4.2	Автоматические тесты	5
5	SAST, SCA и CI/CD	5
5.1	SAST: SpotBugs	5
5.2	SCA: OWASP Dependency-Check	5
5.3	Pipeline	5
6	Выводы	5

1 Цель работы

Разработать защищённое REST API на Java/Spring Boot, реализовать JWT-аутентификацию, базовые меры защиты веб-приложения и интегрировать статический анализ кода и анализ зависимостей в CI/CD pipeline.

2 Описание проекта и API

2.1 Стек технологий

- Язык программирования: Java 17.
- Фреймворк: Spring Boot 3.2.5.
- Система сборки: Gradle 8.7 (Kotlin DSL).
- База данных: PostgreSQL 16, запуск через Docker Compose.
- Аутентификация: JWT с HMAC-SHA256 и BCrypt.

2.2 Endpoints

Метод	Путь	Аутентификация	Описание
POST	/auth/login	Нет	Аутентификация и получение JWT
GET	/api/data	Bearer	Получение защищённых данных
GET	/api/profile	Bearer	Получение профиля пользователя
PUT	/api/profile	Bearer	Обновление профиля

2.3 Примеры запросов

Аутентификация:

POST /auth/login

Content-Type: application/json

```
{ "username": "admin", "password": "admin123" }
```

=> 200 OK

```
{  
  "token": "eyJhbGciOiJIUzI1NiJ9...",  
  "username": "admin",  
  "role": "ADMIN"  
}
```

Получение защищённых данных:

GET /api/data

Authorization: Bearer <token>

=> 200 OK

```
[  
  {"id": 1, "title": "Security Report Q1"},  
  ...  
]
```

```
    {"id": 2, "title": "Vulnerability Scan"},  
    {"id": 3, "title": "Access Logs"}  
]
```

Запрос без токена к `/api/data` возвращает 401 Unauthorized.

3 Реализованные меры защиты

3.1 Защита от SQL-инъекций

Для взаимодействия с базой данных используется Spring Data JPA (Hibernate). Запрос к репозиторию параметризован, а конкатенация строк для формирования SQL в проекте не используется.

```
public interface UserRepository extends JpaRepository<User, Long> {  
    Optional<User> findByUsername(String username);  
}
```

3.2 Защита от XSS

API возвращает данные в формате `application/json`, поэтому значения полей не интерпретируются браузером как HTML или JavaScript. В Spring Security настроены CSP-заголовок и `X-Frame-Options: DENY`. Входные данные профиля проверяются через `@Valid`, `@Email` и `@Size`; HTML-теги в `displayName` запрещены.

Секрет JWT и пароль базы данных в production передаются через переменные окружения.

3.3 Защита аутентификации

Пароли хэшируются с помощью `BCryptPasswordEncoder`. После успешной аутентификации выдаётся JWT-токен с подписью HMAC-SHA256. Фильтр `JwtAuthFilter` проверяет токен в заголовке `Authorization` и устанавливает контекст безопасности для защищённых endpoint'ов. Сессии приложения работают в stateless-режиме.

4 Ход работы и тестирование

4.1 Реализация API

Реализованы контроллеры аутентификации, данных и профиля, DTO для запросов и ответов, JPA-модель пользователя, репозиторий и глобальный обработчик ошибок. Начальные пользователи создаются при запуске приложения.

4.2 Автоматические тесты

Команда `./gradlew test` запускает интеграционные тесты на встроенной H2-базе. Тестами проверяются успешный вход с выдачей JWT, отказ в доступе к `/api/data` без токена, отказ с неверным паролем и отклонение HTML-ввода в профиле.

5 SAST, SCA и CI/CD

5.1 SAST: SpotBugs

Статический анализ исходного кода выполняется задачей `spotbugsMain`. Критических предупреждений не выявлено; исправлены замечания анализа байт-кода и уточнена структура сервиса JWT.

5.2 SCA: OWASP Dependency-Check

Анализ зависимостей выполняется задачей `dependencyCheckAnalyze`. Сборка настроена на контроль критичных уязвимостей с порогом CVSS 7.0.

5.3 Pipeline

В `.github/workflows/ci.yml` на каждый `push` и `pull request` запускаются сборка, SpotBugs, Dependency-Check и тесты. Отчёты SAST и SCA сохраняются как артефакты GitHub Actions.

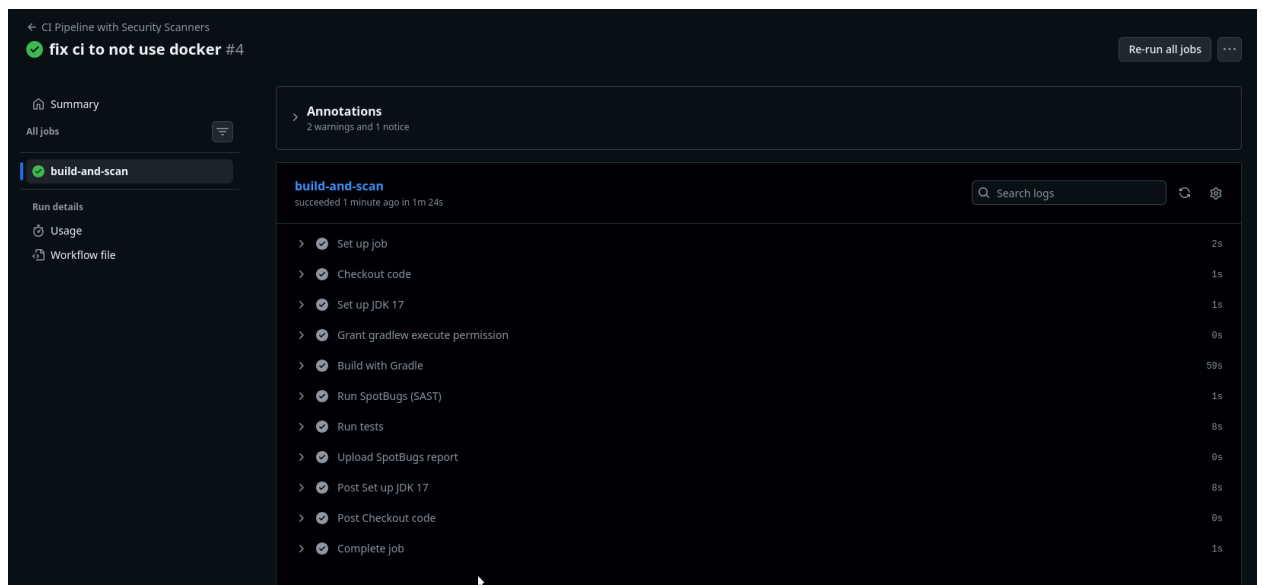


Рисунок 1. Результат выполнения CI/CD pipeline в GitHub Actions.

6 Выводы

Разработано защищённое REST API на Java/Spring Boot с JWT-аутентификацией, хэшированием паролей BCrypt, защитой от SQL-инъекций и XSS.

Реализованы автоматические тесты и CI/CD pipeline с SAST- и SCA-проверками.

Репозиторий: <https://github.com/username/infosec-lab1>.